

Automated Generation of High-Quality Bug Reports for **Android Applications**

Antu Saha
William & Mary
Williamsburg, Virginia, USA
asaha02@wm.edu

Atish Kumar Dipongkor
University of Central Florida
Orlando, Florida, USA
atish.kumardipongkor@ucf.edu

Sam Bennett
William & Mary
Williamsburg, Virginia, USA
srbenett01@wm.edu

Kevin Moran
University of Central Florida
Orlando, Florida, USA
kpmoran@ucf.edu

Andrian Marcus
George Mason University
Fairfax, Virginia, USA
amarcus7@gmu.edu

Oscar Chaparro
William & Mary
Williamsburg, Virginia, USA
oscarch@wm.edu

Abstract

Most defects in mobile applications are visually observable on the device screen. To track these defects, users, testers, and developers must manually submit bug reports, especially in the absence of crashes. However, these reports are frequently ambiguous or inaccurate, often omitting essential components such as the Observed Behavior (OB), Expected Behavior (EB), or Steps to Reproduce (S2Rs). Low-quality reports hinder developers' ability to understand and reproduce defects, delaying resolution and leading to incorrect or unresolvable fixes.

In this paper, we posit that providing specific app-related information (e.g., GUI interactions or specific screens where bugs appear) to LLMs as key points of context can assist in automatically generating clear, detailed, and accurate **OB, EB, and S2Rs**. We built and evaluated a novel approach, **BUGSCRIBE**, that generates bug reports in this way. To support the evaluation, we introduce a unified **quality framework** that defines correctness and completeness dimensions for **OB, EB, and S2Rs**. Using **48 bug reports** from **26 Android apps**, we show that **BUGSCRIBE** produces higher-quality and more accurate components than the original reports and outperforms recent LLM-based baselines. We envision that **BUGSCRIBE** can serve as a practical assistant for testers and developers by enhancing incomplete bug reports with reliable and accurate **OB, EB, and S2Rs**, thereby streamlining bug resolution and improving mobile app quality.

1 Introduction

Most defects in mobile applications are visually observable on the device screen [37]. Since automated mechanisms for detecting and reporting such bugs are typically unavailable—especially in the absence of explicit oracles like crashes—users, testers, and developers must manually submit bug reports (BRs) during app use or testing [64]. Bug reports enable developers to understand, diagnose, reproduce, and ultimately correct the issues [14] and generally

contain natural language descriptions, sometimes accompanied by screen captures, replay videos, or execution traces.

A high-quality bug report should include detailed and unambiguous descriptions of at least three essential components [14, 76]: the **Observed Behavior (OB)**, the **Expected Behavior (EB)**, and the **Steps to Reproduce (S2Rs)**. The OB describes the faulty behavior of the application, the EB specifies how users expect the app should behave, and the S2Rs provide instructions that allow developers to reproduce the problem and create test cases to validate fixes.

Unfortunately, bug reports are often submitted with incomplete, ambiguous, and/or inaccurate descriptions, sometimes omitting the EB or S2Rs entirely [22]. Low-quality reports hinder developers' ability to understand the defects [17], cause delays in resolution [33, 75], and may even lead to reopened issues [75], or unresolvable defects [33]. Writing complete, clear, and detailed bug reports with correct information is essential for streamlining issue resolution and improving app quality.

However, constructing high-quality bug reports is challenging for both developers and end users, particularly for UI-centric software such as mobile apps [45, 64]. The challenge stems from difficulty in understanding and describing program behavior at an appropriate level of abstraction. End users, for example, often rely on a feature-oriented understanding of an app and may describe observed faulty behavior using ambiguous or overly high-level language, sometimes omitting critical details. In contrast, producing unambiguous and complete reports requires a detailed understanding of UI-centric aspects of an app (e.g., UI components, screen flow, allowable user interactions, and supported features), as well as knowing how and when to use this information to *precisely* describe defective behavior. Ultimately, the core challenge of high-quality bug reporting lies in extracting and relating UI-anchored execution information to the observed defect.

Prior work has proposed techniques to improve bug reporting, including annotating problematic S2Rs [17, 43], detecting missing OB/EB/S2Rs [15, 62], providing feedback on report quality [17, 43], and developing interactive reporting tools [29, 46, 63, 64]. However, these approaches do not automatically enhance existing reports; instead, they offer guidance and leave users responsible for making corrections. More recent work leverages large language models (LLMs) to restructure unstructured bug summaries into well-formed reports [11] and to generate missing components (e.g., EB) [15]. While promising, these methods still cannot assess or

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://www.acm.org/permissions).
Conference'17, Washington, DC, USA

© 2026 ACM.
ACM ISBN #---#-----#---#
<https://doi.org/10.1145/3691148>

ensure the quality of generated bug content based on how well it reflects concrete app information, such as UI execution paths and associated metadata. As a result, these methods may still produce incomplete or inaccurate reports; for instance, S2Rs that describe generic interaction sequences unrelated to the user’s actual GUI actions, or OB descriptions that inaccurately specify the system’s buggy state (see Section 5.4). Such inaccuracies can mislead developers, hindering bug understanding, diagnosis, and reproduction. Our **key insight** is that extracting and selectively supplying app UI information most relevant to each bug report component (i.e., **OB, EB, and S2Rs**) enables LLMs to generate bug reports that are both complete and precise by connecting natural language descriptions of a bug in a report to UI-based program components, overcoming the limitations of prior approaches.

In this paper, we introduce BUGSCRIBE, an LLM-powered approach that automatically generates high-quality bug reports from existing reports. BUGSCRIBE is designed to produce detailed and accurate bug report components (**OB, EB, and S2Rs**) by combining various app-related information with the reasoning and language generation capabilities of an LLM. BUGSCRIBE leverages four types of app-related information: (1) OB, EB, and S2R sentences from the original report; (2) GUI-level user interactions that capture app usage; (3) the app screens on which those interactions occur; and (4) the screen suspected to contain the defect. BUGSCRIBE automatically extracts or infers this information. Specifically, BUGSCRIBE constructs a graph-based execution model in which nodes represent app screens and edges represent GUI interactions, enriched with UI metadata such as layouts, widgets, and events. It then (i) identifies OB, EB, and S2R sentences in the report using LLM-based semantic reasoning, and (ii) retrieves relevant screens, interactions, and the likely buggy screen from the execution model via LLM-based analysis. Finally, BUGSCRIBE leverages LLM reasoning to align report text with execution data to infer a plausible reproduction scenario and generate accurate, detailed OB, EB, and S2R descriptions. BUGSCRIBE leverages and augments existing techniques for app execution modeling and buggy screen localization, and introduces novel zero-shot, task-decomposition prompts [28, 38] to (1) translate complex execution data into natural language suitable for LLM reasoning, and (2) generate high-quality bug report components (see Section 3).

We used a data-driven methodology to identify the best combinations of app-specific information for generating each component: **OB, EB, and S2Rs** (Section 4). We systematically evaluated combinations of the four information types used as context for the LLM, generating bug descriptions for 10 existing bug reports across 9 Android apps spanning diverse domains and defect types. Based on manually curated ground truth and the bug report quality model we developed (Section 2), we assessed which combinations produced the highest-quality components. The quality model builds on an existing S2R quality model [17] and extends it to include OB and EB descriptions and their constituents. The results show that different combinations of specific app-related information yield the best performance for each component.

We further evaluated BUGSCRIBE on a dataset of **48 bug reports** from **26 Android apps**, with corresponding ground truth OB, EB, and S2Rs (Section 5), collected from prior work [17, 56]. Using our bug report quality model, we compare BUGSCRIBE against the original bug reports, a recent state-of-the-art LLM-based approach, U2SBR [11],

which generates structured reports from bug summaries, and two LLM baselines that do not use app-specific information as context. Our results show that BUGSCRIBE produces more detailed, complete, and accurate S2Rs than the baselines and the original reports (**44.1% to 82.3%** relative improvement) and generates OB and EB descriptions that are more detailed and accurate than the original reports and the LLM baselines (**3.8% to 35.2%** relative improvement).

In summary, the paper makes the following contributions:

- BUGSCRIBE, an LLM-powered approach that leverages dynamic app analysis, app-related information, and zero-shot, task decomposition prompting to generate detailed, complete, and accurate OB, EB, and S2R descriptions from existing bug reports.
- A data-driven evaluation showing the benefits of combining multiple types of specific app information elements to generate high-quality OB, EB, and S2R descriptions.
- An empirical evaluation showing that BUGSCRIBE improves the quality of original bug reports and generates higher-quality reports compared to three state-of-the-art LLM-based baselines.
- A replication package that includes a manually curated dataset of **58 bug reports** with high-quality ground truth for OB, EB, and S2Rs, along with BUGSCRIBE’s source code, documentation, and experimental infrastructure to facilitate the reproduction and validation of our results [8], and to support future research.

2 Bug Report Quality Model

Our quality model defines quality categories for the three main bug report components: the OB, EB, and S2Rs. A high-quality bug report must contain complete, detailed, and accurate descriptions of these components. The model targets reports of GUI-based applications, in particular Android apps.

S2R Quality Model. The S2Rs in a high-quality bug report are an enumerated list of GUI interactions that a user performs on the app sequentially to reproduce the bug. Each step should be atomic: it should represent a single GUI action (e.g., a tap, type, or swipe) on a particular GUI component (e.g., button or text field) located in an app screen (e.g., an activity, window, or pop-up dialog). An example of high-quality S2R is “On the main screen, tap the ‘More options’ button”. We adapt the **S2R quality model** from prior work [17] and define the following categories:

- **Correct Step (CS):** the step corresponds to a specific GUI interaction in the application.
- **Ambiguous Step (AS):** the step corresponds to multiple interactions on GUI components.
- **Extra Step (ES):** the step is not required to reproduce the bug.
- **Missing Step (MS):** a GUI interaction required to reproduce the bug is missing from the report.

Compared to the original quality model [17], we include the Extra Step (ES) category because automated approaches may generate steps that are unnecessary to reproduce a bug, and we discarded the Vocabulary Mismatch (VM) category [17] because we expect LLMs to always generate well-written steps that map to GUI interactions—we also did not observe this quality issue in our experiments. An example of an *Ambiguous* S2R would be “Restore from backup” (see fig. 1) because multiple individual steps are required to complete the data restoration in the app.

OB Quality Model. The OB in a high-quality report should be a clear, detailed, and self-contained description of the bug and should specify three main information elements: (1) the *buggy app behavior* (the observed app misbehavior), (2) a *buggy screen reference* (the screen where the user observed the bug), and (3) the *triggering GUI interaction* (the interaction that triggers the buggy behavior). An example of a high-quality OB is “The app crashes if I tap the ‘Restore from backup’ option on the pop-up menu”. Our model defines four quality attributes for each of the OB elements. A **quality OB element** can be:

- **Correct:** the element corresponds to plausible buggy app behaviors, screens, or GUI interactions.
- **Incomplete:** the element lacks information to fully map it to plausible buggy app behavior, screens, or GUI interactions.
- **Ambiguous:** the element contains ambiguous or generic information for mapping it specifically to plausible buggy app behavior, app screens, or GUI interactions.
- **Incorrect:** the element describes information that does not map to plausible buggy app behaviors, screens, or GUI interactions.
- **Missing:** the element is not provided in the OB description.

For example, in the OB “The app crashes if I tap the ‘Restore from backup’ option”, the *buggy screen reference* is *Missing*, and if it is written as “The app does not work if I tap the ‘Restore from backup’ option”, it would also contain an *Ambiguous buggy app behavior*, and if it was written as “The app does not work if I ‘Restore from backup’”, it would also contain an *Incomplete triggering GUI interaction*.

EB Quality Model. The EB in a high-quality report should be a clear, detailed, and self-contained description of the *intended behavior* (the expected app behavior), and it is tied to the OB as it describes the opposite (or an alternative) to the OB’s buggy behavior. For example, for the OB “The app crashes if I tap the ‘Restore from backup’ option on the pop-up menu”, the EB should be “The app should successfully restore the backup and display a confirmation dialog instead of crashing”. We define the same five quality attributes as for the OB elements (*i.e.*, correct, incomplete, ambiguous, missing, and incorrect). For instance, an EB with an *Ambiguous intended behavior* would be “The app should work”.

Motivating Example. Figure 1 shows bug report #35 submitted for the ATimeTracker app [1], which allows users to track the time of their daily activities. This report is unstructured and contains an OB with **missing EB** (*i.e.*, the intended behavior is missing) and is missing S2Rs. The OB describes a correct buggy behavior (“*the app crashes with...*”) but contains an **incomplete triggering GUI interaction** (“*trying to restore a backup*”). The report contains an **ambiguous S2R** (“*after selecting the menu item*”).

Figure 1 also shows the bug report that our approach, BUGSCRIBE, generates. Unlike the original bug report, this report is structured and provides complete and detailed OB, EB, and S2R descriptions. For example, the OB includes *correct* descriptions of the buggy behavior (“*...the app crashes with...*”), the triggering app screen (“*On the extended options popup menu...*”), and the triggering GUI interaction (“*... the user taps ‘Restore from backup’...*”). The S2Rs describe a complete reproduction scenario with atomic steps that correspond to specific GUI interactions (see fig. 1.a).

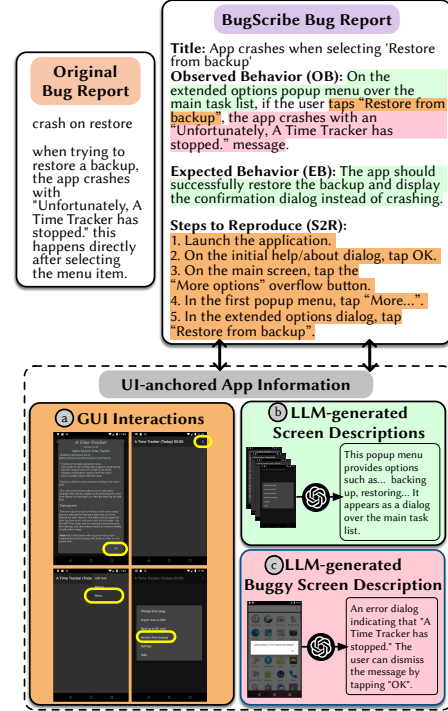


Figure 1: Bug report #35 of the Time Tracker app [1], and the corresponding BUGSCRIBE report. Pieces of the report are linked to: (i) GUI interaction data (orange), (ii) LLM-generated descriptions of bug-related screens (green), and (iii) LLM-generated description(s) of the buggy screen(s) (red).

3 BUGSCRIBE: Generating Quality Bug Reports

We introduce BUGSCRIBE, an LLM-based automated approach that receives an existing bug report and produces a high-quality report that more effectively describes the equivalent defect from the original report. BUGSCRIBE leverages dynamic app analysis, LLM-based language reasoning/generation, and uses app-related information as context for the LLM to generate bug reports with clear, detailed, and accurate OB, EB, and S2Rs. Based on these components, *BUGSCRIBE* also generates a one-line bug summary as bug report title and adds environment details described in the original bug report, including mobile device information and Android version.

In its current version, BUGSCRIBE uses GPT-5.4 [50], a state-of-the-art reasoning LLM. We selected this model because: (1) it offers reasoning of multi-modal information, as our problem involves reasoning about app execution information, app screen data, and natural language and (2) it is a frontier model with state-of-the-art performance in reasoning, coding, and tool use [7]. In BUGSCRIBE’s evaluation, described in Section 5, we instantiate BUGSCRIBE with CLAUDEOPUS-4.6 [13], another state-of-the-art LLM, to assess whether using different frontier models impacts the effectiveness of BUGSCRIBE.

As shown in Figure 2, BUGSCRIBE consists of three main phases:

- (1) **App Execution Model Generation:** BUGSCRIBE constructs a graph-based execution model of the application using both automated and manual app exploration (Section 3.1).

- (2) **Contextual Information Extraction:** BUGSCRIBE extracts and formats app-related information, *e.g.*, GUI interactions and screen descriptions, used as context for the LLM (Section 3.2).
- (3) **Bug Report Generation:** BUGSCRIBE generates high-quality bug reports, including a title, OB, EB, and S2Rs, leveraging the app-specific information and LLM-based reasoning/generation via zero-shot, task-decomposition prompting (Section 3.3).

3.1 App Execution Model Generation

In the first phase, BUGSCRIBE constructs a graph-based app execution model that captures GUI interactions made on app screens, including corresponding app screen metadata such as the UI component hierarchy and events (Figure 2.a). Later, BUGSCRIBE extracts and formats the GUI interactions and the screen information from the execution model as context for the LLM to generate high-quality bug reports (Sections 3.2 and 3.3). We leveraged prior work that uses graph-based representations and dynamic app execution strategies to implement this phase [56, 64].

Specifically, BUGSCRIBE constructs an app execution model as a directed graph $G = (V, E)$, where the nodes V correspond to the unique GUI screens of the app and the edges E capture the possible user interactions with GUI components on those screens. Each screen (*i.e.*, node) is defined by its hierarchy of components and layouts; two screens are treated as distinct nodes if their component hierarchies differ. An interaction (*i.e.*, edge) is represented as a tuple (v_x, v_y, e, c) , where c is a GUI component on screen v_x (*e.g.*, a button or text field), e is the action performed on c (such as tap, swipe, *etc.*), and the interaction leads to a transition to screen v_y . Edges also contain additional metadata describing the interaction, including the component type, ID, label (*i.e.*, text), and description.

BUGSCRIBE builds the execution model from GUI interaction traces collected through both automated exploration and manual app usage. For automated exploration, BUGSCRIBE uses a modified version of CRASHSCOPE [47, 48], which employs multiple UI exploration strategies to maximize screen and component coverage while collecting screenshots, XML-based UI hierarchies, and UI metadata. Because automated exploration may miss interactions commonly performed by real users, BUGSCRIBE can also incorporate traces from manual usage or testing. For the bug reports studied in this paper, we use traces collected in prior work [43, 56], which capture complete interaction sequences from app launch to feature completion. In practice, such manual traces can be collected throughout app evolution, for example, via user monitoring and record-and-replay during in-house or crowd-sourced testing [27].

3.2 Contextual Information Extraction

BUGSCRIBE annotates the user-submitted bug report to specify three components (OB, EB, and S2Rs) and extracts three app information elements (app screen information, GUI interactions, and buggy screen) as context for the LLM to generate high-quality bug reports.

3.2.1 OB/EB/S2R Sentence Annotation. BUGSCRIBE analyzes the input bug report and identifies the **sentences that express the OB, EB, and S2Rs** (Figure 2.b). We frame this task as a text classification problem in which BUGSCRIBE leverages the LLM to classify each sentence in the report as OB, EB, S2R, or other. BUGSCRIBE first decomposes the report into individual sentences, then provides

them as context to GPT-5.4 via zero-shot, rule-based prompting to obtain sentence labels. The original bug report with labeled sentences is passed to the next phase. This labeling allows BUGSCRIBE to prioritize OB/EB/S2R content during component generation, while retaining the full bug report context, which may include extra information such as environment details (build number, device, operating system, *etc.*), code snippets, or stack traces.

We extended the S2R classification prompt template from Mahmud *et al.* [43] following their data-driven methodology for classifying OB, EB, and S2R sentences. The template includes the definition of OB, EB, and S2R with detailed guidelines to classify all sentences in the bug report. The prompt template and the OB/EB/S2R identification results are found in our replication package [8].

3.2.2 App Screen Description Generation. BUGSCRIBE uses **app screen descriptions** generated by a vision-capable LLM as the next piece of app context. These natural language summaries convey the functionality and purpose of the given app screens and may include functional and visual details about screen layouts, GUI components, and functions offered. An example of an app screen description is: “This popup menu provides options such as changing the date range, exporting to CSV, ... It appears as a dialog over the main task list.” **BUGSCRIBE generates a description for each app screen** in the execution model, which is annotated by a unique ID. The description is generated by the LLM using zero-shot prompting, with screen-level metadata (*e.g.*, layout and UI component information) used as context. This information is expected to help the LLM identify the required interactions to reproduce the bug, used to generate atomic S2Rs, as well as a suspected buggy screen (Section 3.2.4). Prompts for this step can be found in our replication package [8].

3.2.3 GUI Interaction Extraction. **GUI interactions** from the execution model are the building blocks for BUGSCRIBE to generate atomic S2Rs, because the required interactions to reproduce the bug are a subset of all the interactions that can be made on the application. From all the possible GUI interactions, BUGSCRIBE uses the LLM to map the provided S2Rs in the original bug report to a navigable interaction sequence from the model that leads to the buggy behavior (Figure 2.b). More precisely, BUGSCRIBE extracts all the GUI interactions of an application from the execution model and parses their metadata into a list of interactions given as context to the LLM (Figure 2.b). Each GUI interaction from the graph is a tuple that contains (1) the action (*e.g.*, tap), (2) the GUI element that the action operates on (*e.g.*, “OK button”), (3) the ID of the *source screen* where the interaction is performed, and (4) the ID of the *target screen* (the resulting screen after the action is performed). This sequence is then synthesized into descriptions of atomic S2Rs. If the S2Rs are missing, the LLM can infer potential steps based on the OB, EB, other information from the bug report, or from the execution model itself.

3.2.4 Buggy Screen Localization. The **buggy screen** is the app screen on which faulty behavior is displayed. It serves as the stopping point for BUGSCRIBE’s S2R generation phase, as it should be the final screen during bug reproduction. It can also assist in the OB/EB description generation step.

BUGSCRIBE receives the descriptions of all app screens from the *screen description generation* step, and relies on LLM-reasoning of

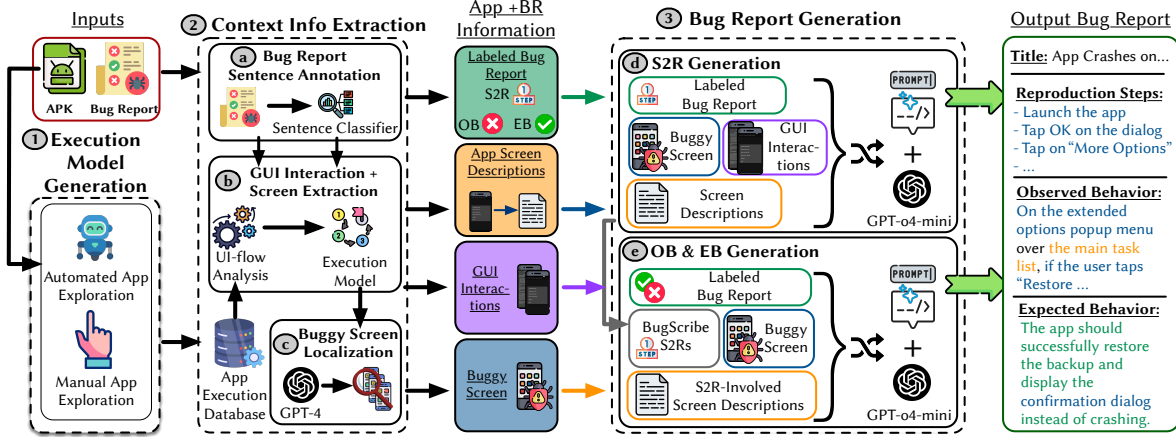


Figure 2: BUGSCRIBE's architecture

the reported bug and the screen description to identify potential buggy screens automatically (Figure 2.c). We formulate this task as a screen retrieval problem, similar to prior work [56], where the goal is to retrieve a ranked list of potential buggy screens for the given bug report from all possible application screens in the execution model. BUGSCRIBE uses a zero-shot prompt with detailed instructions asking the LLM to reason over the input (described below) to return a ranked list of potential buggy screens. BUGSCRIBE then considers the top-ranked screen as the buggy screen, using it as context for bug component generation.

While prior work [56] relies on three deep learning models—e.g., SBERT [53], CLIP [52], and BLIP [40]—we leverage LLMs for this task, which requires careful prompt design and a systematic evaluation of different prompting strategies. Specifically, we designed two experiments for buggy screen localization with two prompts that used different inputs and instructions. The first experiment uses the labeled bug report and the screen descriptions as the input, while the second includes all the app GUI interactions as well. We found that the GUI interactions provides information that helps BUGSCRIBE localize buggy screens more effectively (the correct buggy screen is ranked first in 80% cases, compared to 60% cases), hence, BUGSCRIBE uses all these information elements as context. Our replication package contains the prompt templates and the results of these experiments [8].

3.3 Bug Report Generation

3.3.1 S2R Generation. To generate an improved bug report, BUGSCRIBE first generates atomic S2R descriptions for the input bug report with GPT-5.4 (Figure 2.d). This step uses a zero-shot, task-decomposition prompt to guide the LLM to map the labeled S2R sentences to GUI interaction sequences and infer plausible reproduction scenarios that lead to the buggy screen. The prompt instructs the LLM to identify the most likely reproduction scenario given the described bug and context and generate a detailed, enumerated list of reproduction steps. The prompt starts with the task description, followed by a description of the input (i.e., app-related and report information used as context), the output format, the inputs (structured into well-defined sections), and detailed instructions to complete

the task. To determine the combination of context information elements that lead to the highest-quality generated S2Rs, we employed a data-driven methodology described in Section 4.

3.3.2 OB/EB Generation. The last step in BUGSCRIBE's pipeline generates the OB and EB for the input report using GPT-5.4 (Figure 2.e). This step uses a zero-shot, task decomposition prompt to guide the LLM to map the identified OB and EB from the original bug report to the contextual app information, including the suspected buggy screen, the S2Rs generated in the previous pipeline step, and the source and target screen descriptions for each generated S2R. This step also generates a bug report title and additional environmental information, such as the Android version and device.

The prompt specifies the task description, the inputs (structured into well-defined sections) with their descriptions, detailed instructions to complete the task, and the output format. To determine the combination of contextual information elements that lead to the highest-quality OB and EB, we employed the data-driven methodology described in Section 4.

Finally, BUGSCRIBE assembles the complete bug report by appending the generated output into labeled sections in the following order: title, OB, EB, S2Rs, and additional information.

4 Identifying App-specific Information for Developing BUGSCRIBE

We used a data-driven approach to identify the most effective combinations of app-specific information (e.g., GUI interactions and buggy screen) for BUGSCRIBE's S2R and OB/EB generation. Using a development set of 10 bug reports, we evaluated reports generated by BUGSCRIBE against ground truth OB, EB, and S2Rs that we manually created based on our bug report quality model.

4.1 Development Dataset Construction

4.1.1 Bug Report Collection. We collected the 10 bug reports from the development set from prior work on S2R quality assessment [43]. The bug reports are a subset of Saha *et al.*'s dataset [56] and span diverse bug types and apps. The bugs span nine Android apps from different domains (e.g., finance tracking, file management, multimedia,

etc.) and report various bug types: crashes (3), cosmetic issues (3), output problems (3), and a navigation bug (1). On average, the app execution models associated with the 10 bug reports contain 37.2 UI screens (ranging from 12 to 74 across) and 80.5 GUI interactions (ranging from 19 to 159). The dataset includes OB/EB/S2R sentences, videos that show how to reproduce the bugs, and screenshots of the exercised app screen during automated/manual app exploration and execution. However, it does not include data required to validate report quality (e.g., ground truth OB, EB, and S2R descriptions).

4.1.2 Ground Truth Creation. To create ground truth S2Rs, we first identified a minimal reproduction scenario (a path of GUI interactions) in the app execution model that maps to the S2Rs in the reports. Once identified, we created natural language descriptions of the GUI interactions using GUI interaction metadata (i.e., the action and the interacted GUI component).

For each of the 10 reports, an author read the reported bug, watched the reproduction video, and manually inspected the graph nodes and edges to identify a minimal reproduction scenario. This author created the natural language description of each S2R following the format: [action] [GUI component] (e.g., "tap the OK button"). A second author validated the identified path and S2Rs description, noting potential mistakes and disagreements. The two authors engaged in a discussion session to resolve any issues and reach a consensus to finalize the S2R ground truth. The agreement rate was high, 91.7%, with the most common reason for the disagreement being the misinterpretation of execution model metadata.

During the ground truth creation process, the authors also identified the buggy screen in the execution model: the resulting screen of the last GUI interaction in the reproduction scenario. Based on this screen and the OB/EB in the original reports, one author wrote ground truth OB and EB. For OBs, the format used was: "On [buggy screen reference], if the user [triggering GUI interaction], [buggy app behavior]" (e.g., "On the confirmation dialog, if the user clicks the OK button, the app crashes"). The EB format used was: "[subject] should/shouldn't [intended app behavior]" (e.g., "the app should take me to the previous screen without crashing"). These templates were designed based on the OB/EB discourse patterns identified in prior work [22], which were derived from manual analysis of hundreds of real-life bug reports. A second author validated the ground truth OBs/EBs, and both authors agreed on the final ground truth if changes were needed (e.g., due to mistakes).

4.1.3 Development Dataset Summary. Across the 10 bug reports, ground truth S2Rs include 103 atomic steps (10.3 per bug on average), each corresponding to one specific GUI interaction. For each bug report, we created one OB and EB description with the correct OB/EB information elements.

4.2 Configurations of App-related Information Used as Context to the LLM

4.2.1 Configurations for S2R generation. We designed four configurations that combine three app information elements: GUI interactions, screen descriptions, and the buggy screen.

- (1) **No Info.** uses the original report and no app-related information, allowing us to understand the LLM's ability to generate high-quality S2Rs without any context.

- (2) **Interac.** adds GUI interactions to generate S2Rs, assuming that access to all app interactions enables the LLM to identify and generate the required steps based on the original bug report.
- (3) **Interac.+Scrn.** adds the screen descriptions. Screen descriptions can help the LLM map interaction sequences to the original bug report when generating S2Rs.
- (4) **Interac.+Scrn.+BuggyScrn.** adds the suspected buggy screen. The buggy screen can inform the LLM about the stopping point in the reproduction scenario because it is the final app screen where the bug manifests.

4.2.2 Configurations for OB/EB generation. We designed four configurations combining three app-specific contexts: the buggy screen, BUGSCRIBE-generated S2Rs, and screen descriptions.

- (1) **No Info.** does not use any app context, serving as a bare-minimum baseline for OB/EB generation.
- (2) **Buggy Scrn.** adds the buggy screen description to generate the OB/EB. As the bug should be visible on this screen, it can provide key information such as the OB's buggy screen reference.
- (3) **S2Rs+BuggyScrn.** adds atomic S2Rs generated in the S2R phase to the buggy screen description, helping the LLM identify the triggering GUI interaction.
- (4) **S2Rs+BuggyScrn.+Scrns.** adds screen descriptions of the S2Rs, helping the LLM reason about the steps and identify the triggering GUI interaction.

4.3 Prompt Development Methodology

To evaluate the app-related information configurations, we designed two core prompt templates for the S2Rs and OB/EB generation tasks (described in Section 3.3). We adapted the templates with specific instructions for handling the different app-related information elements, thus creating different instances of the prompts, corresponding to the configurations. The wording and structure of all the prompts are nearly the same. The main difference lies in the inputs (i.e., combinations of app information elements) and specific instructions to handle them. For example, one S2R generation instruction to handle the buggy screen is: "Starting from the initial transition [GUI interaction], i.e., the "open app" transition, your goal is to reach the buggy screen by identifying the most relevant transitions for the user-reported S2Rs." An example instruction to handle GUI interactions is: "The generated S2Rs must form a *valid and complete path* in the transition graph, starting from the initial screen and ending at the screen where the bug occurs."

For each task, one author designed the structured prompt template with detailed instructions for both generation tasks following zero-shot, task-decomposition prompting [28, 38], the OpenAI guidelines [3], and the best practices [58, 59, 70]. For refining and improving the prompts, the author experimented with three bugs from the development set and consulted with the web version of ChatGPT with the example bugs to design an optimized prompt (i.e., we used meta-prompting [73]). A second author reviewed the wording and structure of the designed prompt templates and experimented with the three bug reports. This second author made suggestions to improve the prompts where necessary. Both authors met and discussed the suggestions to finalize the prompt templates.

4.4 Prompt Execution, Metrics, and Evaluation

We executed the eight prompt configurations (four prompts per generation task) using the OpenAI API [2] with the GPT-5.4 model. Note that the latest GPT models do not have temperature control, so we used the model as is. We assessed the consistency of LLM generation across three prompt executions and reported the average bug report component quality using the evaluation metrics described next.

4.4.1 Evaluation Metrics. We evaluated the quality of the generated S2Rs, OBs, and EBs using the bug report quality model described in Section 2, by manually comparing them against ground truth. For S2Rs, we assigned three quality labels: 1) **Correct Step (CS)**: the generated step is found in the ground truth S2Rs (*True Positive or TP*), 2) **Extra Step (ES)**: the generated step is not found in the ground truth S2Rs (*False Positive or FP*), and 3) **Missing Step (MS)**: a ground truth S2R is not found in the generated steps (*False Negative or FN*). Using the quality labels assigned to the S2Rs, we computed *precision*, *recall*, and *F1 score* to measure S2R quality.

For OB and EB evaluation, we assessed each OB/EB information element (e.g., buggy behavior or triggering GUI interaction) using four quality labels (see Section 2 for their definition): **Correct**, **Incomplete**, **Ambiguous**, **Incorrect**, and **Missing**. We counted the number of cases within each quality category for each component element (e.g., the OB's buggy behavior) to measure OB/EB quality.

4.4.2 Methodology for Quality Assessment. We conducted a multi-coder qualitative evaluation to mitigate the potential biases. Two authors independently compared the generated S2Rs/OB/EB with the respective ground truth and assigned quality labels according to the quality model. To ensure reliable and consistent evaluation, we designed detailed evaluation criteria for each bug report component and specific guidelines with instructions (see our replication package [8]). The authors engaged in a discussion session to resolve the disagreements following the criteria to reach a consensus. If the two annotators could not agree, a third annotator evaluated such cases to make a final decision via consensus.

We assessed inter-annotator reliability using observed agreement, Cohen's Kappa (κ) [24] and Krippendorff's Alpha (α) [39], obtaining overall high agreement: 99.5% agreement for S2Rs ($\kappa = 0.98$, almost perfect agreement [68]; $\alpha = 0.99$, reliable agreement [44]) and 87.5% agreement for OBs/EBs ($\kappa = 0.6$, moderate agreement [68]; $\alpha = 0.94$, reliable agreement [44]).

4.5 Results and Analysis

4.5.1 Quality of Generated S2Rs. To assess LLM consistency (producing the same outputs for multiple runs), we compared the evaluation metrics of the configurations across the three runs. We found that all configurations achieved consistent performance across the runs. For instance, the standard deviation of the F1 score in *Interac.+Scrn.+BuggyScrn.* is 0.01, indicating high consistency. Our replication package contains the results of all configurations [8].

Table 1 shows the S2R quality results of the four configurations for S2R generation, averaged across the three executions. All three context-aware configurations substantially outperform the no-information configuration, with F1 increasing from 72.02 (*No Info.*) to 87.33–88.49 across the other setups. Notably, compared to *No Info.*, using GUI interactions yields a large gain (F1 87.33 vs

Table 1: Total # of correct (CS), extra (ES), and missing S2Rs (MS) in the development bug reports (avg. over three runs)

Config	CS (TP)	ES (FP)	MS (FN)	Prec.	Rec.	F1
<i>No Info.</i>	80.7	34.3	28.3	70.15	74.01	72.02
<i>Interac.</i>	95.3	14.0	13.7	87.20	87.46	87.33
<i>Interac.+Scrn.</i>	97.7	15.0	12.3	86.69	88.79	87.73
<i>Interac.+Scrn.+BuggyScrn.</i>	97.3	13.0	12.3	88.22	88.75	88.49

Table 2: Total # of OB/EB elements in the development bug reports (avg. over three runs)

Config.	Correct	Incomp.	Ambig.	Missing	Incor.
<i>No Info.</i>	30.3	5.3	1.7	0	2.7
<i>Buggy Scrn.</i>	32.3	5.3	0	0	2.3
<i>S2Rs+BuggyScrn.</i>	33.3	5.3	0.3	0	1
<i>S2Rs+BuggyScrn.+Scrn.</i>	34.3	4.3	0	0	1.3

72.02), indicating that this context is essential for accurately identifying an associated reproduction path. Adding screen descriptions and/or the buggy screen to GUI interactions leads to small but consistent improvements, with the best performance achieved by *Interac.+Scrn.+BuggyScrn.* (F1 88.49, Precision 88.22, Recall 88.75). While all three context-aware configurations produce a similar number of correct steps (95.3–97.7), the best configuration reduces both extra steps (13.0 vs. 14.0–15.0) and missing steps (12.3 vs. 13.7).

Qualitative analysis of the results shows that including the buggy screen informs the LLM about the final screen where the bug manifests, constraining the reproduction path. Since each interaction has a source and a target screen (see Section 3.2.3), the target screen of the final step should correspond to the buggy screen. Leveraging this constraint allows the LLM to more accurately align the reported steps from the original bug report with the actual interactions, resulting in fewer extra steps, compared to *Interac.+Scrn.*

4.5.2 Quality of Generated OBs/EBs. Table 2 presents the total number of OB/EB elements in the generated 10 bug reports, across the quality categories from our OB/EB quality model, and the four OB/EB generation configurations. All the configurations consistently generated OBs/EBs with similar elements across all three runs (see our replication package for the results of all executions [8]).

Out of 40 OB/EB elements (4 OB/EB elements types $\times$ 10 bug reports), we found that all configurations yield similar results across the quality categories, with correct OB/EB elements ranging from 30.3 (*No Info.*) to 34.3 (*S2Rs+BuggyScrn.+Scrn.*), and consistently low numbers of incomplete (5.3–4.3), ambiguous (1.7–0), and incorrect elements (2.7–1.3). However, incorporating context (particularly the buggy screen, shared across all context-aware configurations) improves element correctness (e.g., 32.3–34.3 vs. 30.3). The best performance is achieved by *S2Rs+BuggyScrn.+Scrn.*, which includes buggy screens, generated S2Rs, and screen descriptions, yielding the highest number of correct elements (34.3) and fewer incomplete ones (4.3). Across all configurations, no OB/EB elements are missing, and results for each OB/EB element (not shown in the table, but found in our replication package [8]) follow the same trends, confirming that BUGSCRIBE effectively leverages contextual information to generate high-quality OB and EB descriptions.

5 BUGSCRIBE's Evaluation

The primary goal of BUGSCRIBE's evaluation is to assess the quality of the generated S2Rs, OBs, and EBs compared to baseline approaches on a larger dataset of 48 bug reports. Given this goal, we answer the following research questions (RQs):

- **RQ₁**: What is the quality of BUGSCRIBE's S2Rs compared to baseline and original reports?
- **RQ₂**: What is the quality of BUGSCRIBE's OBs/EBs compared to baseline and original reports?

5.1 Test Dataset

We evaluated BUGSCRIBE with a dataset of 48 bug reports (*i.e.*, a test set, different from the development set), which enables comparison with baseline techniques (Section 5.2). We collected these bug reports via stratified random sampling across various bug types from 96 reports used in prior work [17, 56]. These 48 reports span 26 Android applications of various domains (web browsing, WiFi diagnosis, finance tracking, *etc.*) and cover different bug types: output issues (18), cosmetic problems (12), crashes (12), and navigation bugs (6). Corresponding app execution models have an average of 43.4 GUI screens (ranging from 5 to 105; median: 27.5) and 117.5 GUI interactions (ranging from 10 to 282; median: 92.5).

These bug reports include OB/EB/S2R sentence labels, but do not include the high-quality bug report ground truth, which is required to evaluate BUGSCRIBE. Hence, we manually constructed the high-quality S2Rs and OB/EB ground truth for each bug report following the data construction methodology described in Section 4.1.2 and Section 4.2.2. Moreover, we extracted and evaluated the four OB/EB information elements and S2Rs found in the original bug report according to our quality model to evaluate how the BUGSCRIBE-generated high-quality bug reports improve over the original reports. As in the development set construction, two authors used a rigorous qualitative methodology to construct the ground truth for the test set. They achieve agreement rates of 89.6% for OB/EB and 93.5% for S2R ground truth construction. In total, we required approximately 304 human hours to construct the dataset, underscoring the substantial effort required to generate ground-truth data for high-quality bug report generation.

In summary, the 48 bug reports include 599 fine-grained S2Rs (12.5 steps per bug), each representing one specific application interaction, *e.g.*, taps, long-taps, *etc.* They also include the high-quality OB/EB ground truth with the correct information elements.

5.2 Baseline techniques

We implemented BUGSCRIBE using two state-of-the-art LLMs, *i.e.*, GPT-5.4 and CLAUDEOPUS-4.6 [13], resulting in two variants: BS-GPT and BSCLAUDE. Both variants use the best context configurations found in our BUGSCRIBE development experiments, for both S2R and OB/EB generation, namely: *Interac.+Scrns.+BuggyScrns.* and *S2Rs+BuggyScrns.+Scrns.*, respectively.

We evaluated both variants against a state-of-the-art bug report improvement technique, U2SBR [11]. We also implemented two baselines, *i.e.*, GPTNO-INFO and CLAUDENO-INFO, which leverage the same LLMs without using any app-specific information.

5.2.1 GPTNO-INFO and CLAUDENO-INFO. These baselines generate bug reports following a similar generation pipeline as BUGSCRIBE, including the same two main prompt templates for S2R and OB/EB generation. However, these prompt templates do not include the placeholders for the app-specific information and the specific instructions to handle the information, *i.e.*, the *No Info.* configuration (see section 4.2 for more details). We designed this baseline to understand how GPT-5.4 and CLAUDEOPUS-4.6 perform for bug report generation without any app-specific information.

5.2.2 U2SBR. Acharya *et al.* [11] proposed an LLM-based approach to transform unstructured bug reports into structured bug reports with explicit OB, EB, and S2Rs. (We refer to this approach U2SBR in this paper.) The original approach uses GPT-4o with three-shot prompting to generate structured bug reports for the given unstructured bug report without any app context. To enable a fair comparison, we replaced its GPT-4o model with GPT-5.4. We implemented this baseline using its companion replication package [4].

5.3 Methodology and Metrics

We ran BUGSCRIBE and all baseline techniques on the 48 bug reports in the test set and generated high-quality bug reports for the given original bug reports. Given that our development experiments found near-perfect consistency in LLM responses under the same inputs across three executions, we executed all the approaches only once in this evaluation. To address RQ₁, we manually compared the generated S2Rs to the ground truth S2Rs following the methodology discussed in Section 4.4.2 and computed precision, recall, and F1 score (Section 4.4.1). To answer RQ₂, we extracted the four information elements from the BUGSCRIBE-generated OBs and EBs and compared them with the ground truth annotations manually (Section 4.4.2). We counted the bug reports with different quality labels (see Section 4.4.1) for each OB/EB element. Overall, the agreement rates between the two evaluators for S2Rs is 97.3% ($\kappa = 0.92$, *i.e.*, almost perfect agreement [68]; $\alpha = 0.99$, *i.e.*, reliable agreement [44]), and for OB/EB is 92.9% ($\kappa = 0.81$, *i.e.*, moderate agreement [68]; $\alpha = 0.97$, *i.e.*, reliable agreement [44]). Our replication package includes the agreement results for each approach [8].

Comparison Protocol. As GPTNO-INFO, CLAUDENO-INFO, and U2SBR do not use application information and are not designed to generate atomic S2Rs, they can produce compound S2Rs (*i.e.*, multiple steps in one S2R sentence), contain different wording than the application vocabulary, or implicitly mention multiple steps. To ensure a fair comparison, we adopted the following strategies. If there are m atomic steps in the generated bug report, and among them n are present in the ground truth as individual steps, we counted n correct steps. Moreover, we considered generated steps with different wording or presentation but similar meaning to the ground truth step as *correct*. If multiple steps are implicitly mentioned, we counted them individually. We also used these strategies to compare the S2R quality of the original bug reports against the ground truth, which is required to compare the reports generated by BUGSCRIBE and the baselines with the original reports using our evaluation metrics. A complete comparison protocol to ensure a fair comparison is found in our replication package [8].

5.4 Results and Analysis

5.4.1 RQ1: Quality of S2R Generation. Table 3 presents the quality results of the S2Rs in the original bug reports and the S2Rs generated by BSGPT, BSCLAUDE, and the three baselines.

BUGSCRIBE vs. Baselines and Original Reports. The table shows that BUGSCRIBE (both BSGPT and BSCLAUDE) clearly outperforms all baselines and the original bug reports across all metrics. In particular, it generates substantially more correct steps (532, 538 vs. 196, 243, 345, 312) while significantly reducing both extra and missing steps (e.g., 57, 67 vs. 75, 356 by U2SBR). This results in large gains in F1 (89.56, 88.27 vs. 49.12–59.66), demonstrating that BUGSCRIBE produces higher-quality S2Rs than all alternatives.

These improvements highlight the importance of app context, which enables the LLM to better map S2Rs to reproduction paths from the app execution model. While baselines without such context (GPTNO-INFO, CLAUDENO-INFO, and U2SBR) still improve over the original bug reports (e.g., F1 61.22, 59.66, 53.00 vs. 49.12), they remain far behind BUGSCRIBE. Across LLMs, BSGPT and BSCLAUDE achieve similar overall performance, with a slight trade-off: BSCLAUDE achieves slightly higher recall (89.82) and more correct steps (538), while BSGPT achieves substantially higher precision (90.32) and fewer extra steps (57), indicating more precise S2R generation.

The lower performance of the baselines stems from the lack of context for S2R generation, as they solely rely on user-submitted reports to generate a possible reproduction path. This leads to either missing steps (356 for U2SBR) or over-generation (183 and 135 extra steps for GPTNO-INFO and CLAUDENO-INFO). For instance, in bug report #84 [10] of the Ultrasonic app, the baselines correctly generate “enter the server URL” but introduce unnecessary steps to reproduce the bug (e.g., “enter the username” and “enter the password”) and miss required interactions (e.g., “Click OK” and Swipe up” before “Click Test Connection”), whereas BSGPT and BSCLAUDE avoid such errors by leveraging app-specific information.

BUGSCRIBE’s superior performance stems from combining multiple app information types: GUI interactions, screen descriptions, and the buggy screen. GUI interactions constrain the LLM to valid user actions, screen descriptions provide source and target context for each interaction, and the buggy screen anchors the final state of the reproduction scenario. Together, these signals enable the LLM to identify valid execution paths from the initial screen to the buggy screen and generate accurate S2Rs aligned with the reported bug.

Analysis of Failed Cases. Despite substantial improvements, BSGPT and BSCLAUDE still miss 67 and 61 steps and produce 57 and 82 extra steps, respectively. Our qualitative analysis of these cases reveals three main causes: (1) multiple feasible paths in the app execution model, (2) very low-quality user reports with incomplete or ambiguous steps, and (3) inaccuracies in buggy screen localization. For example, bug report #1402 [9] (Phimp.me app) contains only one vague step (“Try with different images in share activity”), leading BSGPT and BSCLAUDE to generate only 6 and 8 correct steps (vs. 13 ground truth steps). With insufficient information, BUGSCRIBE tends to produce a shorter but suboptimal path. Addressing this limitation may require combining LLM reasoning with path-finding techniques and human-in-the-loop approaches [64] to better identify correct interaction paths associated with the bug.

Table 3: Total # of correct (CS), extra (ES), and missing S2Rs (MS) in the test set bug reports

Approach	CS (TP)	ES (FP)	MS (FN)	Prec.	Rec.	F1
Original BR	196	3	403	98.49	32.72	49.12
U2SBR	243	75	356	76.42	40.57	53.00
GPTNO-INFO	345	183	254	65.34	57.60	61.22
CLAUDENO-INFO	312	135	287	69.80	52.09	59.66
BSGPT	532	57	67	90.32	88.82	89.56
BSCLAUDE	538	82	61	86.77	89.82	88.27

Table 4: Total # of OB/EB elements in the 48 test set reports

OB/EB Element	Approach	Cor.	Incomp.	Ambig.	Miss.	Incor.
Buggy Behavior (OB)	Original BR	46	1	0	1	0
	U2SBR	45	3	0	0	0
	GPTNO-INFO	43	3	0	0	2
	CLAUDENO-INFO	46	1	0	0	1
	BSGPT	46	0	0	0	2
	BSCLAUDE	46	1	0	0	1
Triggering GUI Interaction (OB)	Original BR	28	3	1	16	0
	U2SBR	21	10	2	11	4
	GPTNO-INFO	36	7	0	0	5
	CLAUDENO-INFO	36	10	1	0	1
	BSGPT	43	2	0	0	3
	BSCLAUDE	37	9	0	1	1
Buggy Screen Reference (OB)	Original BR	12	3	1	32	0
	U2SBR	10	1	5	23	9
	GPTNO-INFO	22	4	2	12	7
	CLAUDENO-INFO	30	0	5	6	7
	BSGPT	29	5	1	0	11
	BSCLAUDE	35	0	4	2	7
Intended Behavior (EB)	Original BR	36	1	1	10	0
	U2SBR	47	0	1	0	0
	GPTNO-INFO	41	5	0	0	2
	CLAUDENO-INFO	47	1	0	0	0
	BSGPT	41	5	0	0	2
	BSCLAUDE	47	1	0	0	0
Total	Original BR	122	8	3	59	0
	U2SBR	123	14	8	34	13
	GPTNO-INFO	142	19	2	12	16
	CLAUDENO-INFO	159	12	6	6	9
	BSGPT	158	12	1	0	19
	BSCLAUDE	165	11	4	3	9

5.4.2 RQ2: Quality of OB/EB Generation. Table 4 shows the number of OB/EB information elements in the original bug reports and the reports generated using different approaches. The table shows results for each individual OB/EB element type, with elements categorized by their quality category.

BUGSCRIBE vs. Baselines and Original Reports. The results vary depending across the individual OB/EB elements. (Note that as U2SBR was not designed to include triggering GUI interaction and buggy screen reference, we abstain from directly comparing BUGSCRIBE with U2SBR for these two elements.) For *Buggy Behavior (OB)*, all approaches perform similarly (e.g., 46 correct elements, out of 48), indicating limited impact of app-specific context on generating this element. This can be explained by the fact that the buggy behavior is typically present in original reports. In contrast, for the OB’s *Triggering GUI Interaction*, BUGSCRIBE performs better, especially BSGPT, generating more correct elements (43 by BSGPT) than the baselines and reducing missing or incomplete cases. For the OB’s *Buggy Screen Reference*, results are mixed, but BSCLAUDE

achieves the highest performance (35 correct, 2 missing elements), while BSGPT still improves over original reports (29 vs. 12). For *Intended Behavior (EB)*, BUGSCRIBE performs comparably to the baselines, suggesting that context is less important, as intended behavior can often be inferred from the buggy behavior, which appears in 46 of the original reports.

Overall, BUGSCRIBE improves substantially over original bug reports, increasing the number of correct elements (e.g., 158–165 vs. 122) and reducing the number of missing ones (0–3 vs. 59), though with slight increases in incomplete and incorrect cases. Compared to the baselines, BSCLAUDE achieves the best overall balance (165 correct, 3 missing), outperforming all approaches across most metrics, while BSGPT also performs strongly but with slightly more incorrect elements. Notably, CLAUDE-INFO performs comparatively (with 159 correct elements), approaching BSGPT, suggesting that while context is beneficial, the choice of LLM also influences performance.

Analysis of Failed Cases. Manual inspection of the incomplete and incorrect triggering GUI interactions generated by BSGPT/BSCLAUDE reveals that extra or unnecessary reproduction steps in the original bug report can misguide BUGSCRIBE. For example, bug report #1066 [6] (Focus-android app) contains the line: “Repeat moving cursor and clicking system status bar...”, where the user attempted to circumvent the bug. This operation is not a part of the reproduction steps; however, BUGSCRIBE consider this as the triggering GUI interaction, resulting in an incomplete/incorrect element.

We analyzed the ambiguous buggy screen references generated by BUGSCRIBE and found that, although imprecise, they refer to the correct buggy screens. These ambiguities arise because BUGSCRIBE often uses generic, functionality-based names rather than the exact UI wording. For instance, in bug report #154 for the Camfahrplan app [5], the correct buggy screen is “*schedule screen*”, whereas BSCLAUDE refers to it as “*event details screen*”. This discrepancy stems from limitations in the app metadata used to build the app execution model, which may omit or differ from the user-visible screen names, prompting BUGSCRIBE to infer names from the UI screen descriptions used as context. To address this, our future work will incorporate visual information from UI screenshots via multimodal models to improve screen naming. Besides ambiguous cases, incorrect buggy screen were primarily generated because the buggy screen localization phase failed to identify the correct buggy screen. Our future work will experiment with the inclusion of multiple buggy screen suggestions, i.e., not only leverage the most suspicious screen detected by the localization phase as context.

6 Threats to Validity

Construct Validity. The manual construction of high-quality ground truth OB, EB, and S2Rs and the manual evaluation of generated bug reports may introduce subjectivity. To mitigate this, two authors performed these tasks using well-defined, replicable procedures. In-depth discussion sessions were conducted to resolve the misunderstandings and disagreements. We further quantified the consistency of their work by computing inter-rater agreement, which was consistently very high across all cases, reinforcing the reliability of BUGSCRIBE’s development and evaluation.

Internal Validity. The performance of BUGSCRIBE can vary depending on the wording and structure of the prompt templates.

To mitigate this, we systematically designed the templates following the best practices [58, 59, 70] and evaluated the prompts with different configurations on the development dataset. LLM non-determinism may cause results to fluctuate across executions. We addressed this by running our development experiments three times, reporting consistent results across runs. While the baselines were not designed to produce atomic S2Rs, we applied consistent and transparent rules to ensure a fair comparison with BUGSCRIBE, documenting our procedure to ensure reproducibility.

External Validity. The results may or may not generalize to bug reports and apps outside from our datasets. However, these datasets contain bug reports of various bug types (e.g., crashes, UI issues, navigation problems, etc.) and 26 Android apps of various kinds (e.g., web browsing, WiFi diagnosis, finance tracking, etc.) and sizes. BUGSCRIBE is developed using a distinct dataset, i.e., a development set of 10 bug reports from nine apps of various domains (e.g., file management, multimedia, etc.) and different bug types. BUGSCRIBE’s evaluation using a different data set (the test set) and the fact that the quality results between both datasets are similar gives confidence in the generalizability of the results.

7 Related Work

Bug reports have been studied for various purposes, such as understanding bug resolution process [55], bug management [12, 42, 54, 56, 77], detecting duplicates [18, 21, 25, 34, 71, 74], predicting priority and severity [35, 66, 67], localizing faulty code [19, 20, 23, 32], identifying solution discussions [57], classifying bug types [16, 61], reproducing bugs [30, 31, 69], and more.

Bug Report Quality Assessment and Interactive Bug Reporting. Past research has proposed methods to assess bug report quality from multiple angles, including lexical analysis (e.g., readability) [26, 41, 76] and presence of bug information (e.g., patches and screenshots) [17, 76]. The most related to work is by Chaparro *et al.* [17], who proposed EULER to evaluate S2R quality via heuristics that map keywords in reports to app UI information. More recently, Mahmud *et al.* [43] introduced ASTROBR, which uses LLMs to assess S2R quality. Imran *et al.* [36] aimed to enhance incomplete reports by generating follow-up clarification questions to developers.

Other work has investigated interactive bug reporting methods. Song *et al.* introduced the chabot BURT [63, 64] to guide reporters during submission, verifying bug information quality in real time and providing improvement suggestions. Moran *et al.* presented FUSION [46], which enabled reporters to construct structured reports by selecting actions and GUI components from drop-down lists. Fazzini *et al.* proposed EBUG [29], which extends FUSION [46] by suggesting S2Rs to reporters in addition to dropdown-based selections.

These methods provide feedback on problematic bug report content but leave corrections to end-users, rather than automatically improving reports via LLM-provided app context as BUGSCRIBE does.

Bug Report Quality Improvement. Prior research on enhancing the bug report quality has primarily focused on detecting missing components (e.g., OB/EB/S2R) and generating structured reports by leveraging LLMs. For instance, Acharya *et al.* [11] proposed an LLM-based technique to transform unstructured bug summaries into structured reports with explicit OB, EB, and S2Rs. Bo *et al.* [15] introduced ChatBR, which first identifies components completely

missing in the report using a fine-tuned BERT classifier and then employs an LLM to generate the missing content. Compared to these approaches, BUGSCRIBE's primary contribution is a novel, integrated architecture that selectively extracts and combines various kinds of app-specific information tailored to each bug report component. By combining dynamic analysis with LLM reasoning through novel prompt designs, BUGSCRIBE produces OB, EB, and S2Rs that are explicitly grounded in concrete application behavior, resulting in more accurate and complete bug reports (as the evaluation shows). Other work has used LLMs to analyze bug reports and recommend relevant screenshots [65] and user reviews [51], generate bug reports based on chat conversions [60], classify report sentences into templates [72], and generate issue report templates [49].

8 Conclusions

We learned that by leveraging different types of app-related information (i.e., GUI interactions, screen metadata, and buggy screen references), LLMs can be used to generate high-quality bug reports better than existing, more generic approaches, which rely only on the user-provided textual descriptions of the bug. Our tool, BUGSCRIBE, generates bug reports with higher quality than those generated by a state-of-the-art baseline technique and the original user-written bug reports.

9 Data Availability Statement

We made all study artifacts, including the dataset, source code, and documentation, publicly accessible [8].

References

- [1] 2025. ATimeTracker Bug Report 35 - <https://github.com/netmackan/ATimeTracker/issues/35>.
- [2] 2025. OpenAI API - <https://openai.com/api/>.
- [3] 2025. Prompt engineering - <https://platform.openai.com/docs/guides/prompt-engineering>.
- [4] 2025. U2S-BR Replication Package. https://github.com/GindeLab/Ease_2025_AI_model.
- [5] 2026. Camfahrplan Bug Report 154 - <https://github.com/tuxmobil/CampFahrplan/issues/154>.
- [6] 2026. Focus-android Bug Report 1066 - <https://github.com/mozilla-mobile/focus-android/issues/3152>.
- [7] 2026. Introducing GPT-5.4 - https://openai.com/index/introducing-gpt-5-4/?utm_source=chatgpt.com.
- [8] 2026. Online Replication Package. <https://zenodo.org/records/19341838>.
- [9] 2026. Phimp.me Bug Report 1402 - <https://github.com/fossasia/phimpme-android/issues/1039>.
- [10] 2026. Ultrasonic Bug Report 84 - <https://github.com/ultrasonic/ultrasonic/issues/187>.
- [11] Jagrit Acharya and Gouri Ginde. 2025. Can We Enhance Bug Report Quality Using LLMs?: An Empirical Study of LLM-Based Bug Report Generation. *arXiv preprint arXiv:2504.18804* (2025).
- [12] Ahmed Adnan, Antu Saha, and Oscar Chaparro. 2025. SPRINT: An Assistant for Issue Report Management. *Proceedings of the 22nd IEEE/ACM International Conference on Mining Software Repositories (MSR'25)* (2025).
- [13] Anthropic. 2025. Claude Opus 4.6. <https://platform.claude.com/docs/en/about-claude/models/overview>. Accessed: 2026-03-26.
- [14] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What Makes a Good Bug Report?. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Atlanta, Georgia). 308–318.
- [15] Lili Bo, Wangjie Ji, Xiaobing Sun, Ting Zhang, Xiaoxue Wu, and Ying Wei. 2024. ChatBR: Automated assessment and improvement of bug report quality using ChatGPT. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA). 1472–1483.
- [16] Gemma Catolino, Fabio Palomba, Andy Zaidman, and Filomena Ferrucci. 2019. Not all bugs are the same: Understanding, characterizing, and classifying bug types. *Journal of Systems and Software* 152 (2019), 165–181.
- [17] Oscar Chaparro, Carlos Bernal-Cárdenas, Jing Lu, Kevin Moran, Andrian Marcus, Massimiliano Di Penta, Denys Poshyvanyk, and Vincent Ng. 2019. Assessing the Quality of the Steps to Reproduce in Bug Reports. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Tallinn, Estonia). 86–96.
- [18] Oscar Chaparro, Juan Manuel Florez, and Andrian Marcus. 2016. On the vocabulary agreement in software issue descriptions. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution*. IEEE, 448–452.
- [19] Oscar Chaparro, Juan Manuel Florez, and Andrian Marcus. 2017. Using observed behavior to reformulate queries during text retrieval-based bug localization. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 376–387.
- [20] Oscar Chaparro, Juan Manuel Florez, and Andrian Marcus. 2019. Using bug descriptions to reformulate queries during text-retrieval-based bug localization. *Empirical Software Engineering* 24 (2019), 2947–3007.
- [21] Oscar Chaparro, Juan Manuel Florez, Unnati Singh, and Andrian Marcus. 2019. Reformulating queries for duplicate bug report detection. In *2019 IEEE 26th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 218–229.
- [22] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting Missing Information in Bug Descriptions. In *Proceedings of the 11th Joint Meeting on the Foundations of Software Engineering (ESEC/FSE'17)*. 396–407.
- [23] Oscar Chaparro and Andrian Marcus. 2016. On the reduction of verbose queries in text retrieval based software maintenance. In *Proceedings of the 38th International Conference on Software Engineering Companion*. 716–718.
- [24] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46.
- [25] Nathan Cooper, Carlos Bernal-Cárdenas, Oscar Chaparro, Kevin Moran, and Denys Poshyvanyk. 2021. It takes two to tango: Combining visual and textual information for detecting duplicate video-based bug reports. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 957–969.
- [26] Bogdan Dit and Andrian Marcus. [n.d.]. Improving the readability of defect reports. In *Proceedings of the 2008 International Workshop on Recommendation Systems for Software Engineering (RSSE)*. 47–49.
- [27] Mingzhe Du, Shengcheng Yu, Chunrong Fang, Tongyu Li, Heyuan Zhang, and Zhenyu Chen. 2022. SemCluster: a semi-supervised clustering tool for crowd-sourced test reports with deep image understanding. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1756–1759.
- [28] Dheeru Dua, Shivanshu Gupta, Sameer Singh, and Matt Gardner. 2022. Successive prompting for decomposing complex questions. *arXiv preprint arXiv:2212.04092* (2022).
- [29] Mattia Fazzini, Kevin Moran, Carlos Bernal-Cárdenas, Tyler Wendland, Alessandro Orso, and Denys Poshyvanyk. 2023. Enhancing Mobile App Bug Reporting via Real-Time Understanding of Reproduction Steps. *IEEE Transactions on Software Engineering* 49, 3 (March 2023), 1246–1272.
- [30] Mattia Fazzini, Martin Prammer, Marcelo d'Amorim, and Alessandro Orso. 2018. Automatically translating bug reports into test cases for mobile apps. In *Proceedings of the 27th International Symposium on Software Testing and Analysis (ISSTA'18)*. 141–152.
- [31] Sidong Feng and Chunyang Chen. 2024. Prompting Is All You Need: Automated Android Bug Replay with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering* (Lisbon, Portugal).
- [32] Juan Manuel Florez, Oscar Chaparro, Christoph Treude, and Andrian Marcus. 2021. Combining query reduction and expansion for text-retrieval-based bug localization. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 166–176.
- [33] Philip J. Guo, Thomas Zimmermann, Nachiappan Nagappan, and Brendan Murphy. 2010. Characterizing and predicting which bugs get fixed: an empirical study of Microsoft Windows. In *Proceedings of the 32nd International Conference on Software Engineering*. 495–504.
- [34] Jianjun He, Ling Xu, Meng Yan, Xin Xia, and Yan Lei. 2020. Duplicate bug report detection using dual-channel convolutional neural networks. In *Proceedings of the 28th International Conference on Program Comprehension*. 117–127.
- [35] Zijie Huang, Zhiqing Shao, Guisheng Fan, Huiqun Yu, Kang Yang, and Ziyi Zhou. 2022. Bug report priority prediction using developer-oriented socio-technical features. In *Proceedings of the 13th Asia-Pacific symposium on internetware*. 202–211.
- [36] Mia Mohammad Imran, Agnieszka Ciborowska, and Kostadin Damevski. 2021. Automatically Selecting Follow-up Questions for Deficient Bug Reports. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. 167–178.
- [37] Jack Johnson, Junayed Mahmud, Tyler Wendland, Kevin Moran, Julia Rubin, and Mattia Fazzini. 2022. An empirical investigation into the reproduction of bug reports for android apps. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 321–322.

- [38] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2022. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406* (2022).
- [39] Klaus Krippendorff. 2018. *Content analysis: An introduction to its methodology*. Sage publications.
- [40] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. 2022. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *ICML'22*.
- [41] Erik Linstead and Pierre Baldi. 2009. Mining the coherence of GNOME bug reports with statistical topic models. In *Proceedings of the 6th IEEE International Working Conference on Mining Software Repositories*. 99–102. <https://doi.org/10.1109/MSR.2009.5069486>
- [42] Junayed Mahmud, Nadeeshan De Silva, Safwat Ali Khan, Seyed Hooman Mostafavi, S M Hasan Mansur, Oscar Chaparro, Andrian (Andi) Marcus, and Kevin Moran. 2024. On Using GUI Interaction Data to Improve Text Retrieval-based Bug Localization. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering* (Lisbon, Portugal).
- [43] Junayed Mahmud, Antu Saha, Oscar Chaparro, Kevin Moran, and Andrian Marcus. 2025. Combining language and app ui analysis for the automated assessment of bug reproduction steps. *arXiv preprint arXiv:2502.04251* (2025).
- [44] Giacomo Marzi, Marco Balzano, and Davide Marchiori. 2024. K-Alpha calculator-krippendorff's alpha calculator: a user-friendly tool for computing krippendorff's alpha inter-rater reliability coefficient. *MethodsX* 12 (2024), 102545.
- [45] Kevin Moran, Mario Linares-Vásquez, Carlos Bernal-Cárdenas, and Denys Poshyvanyk. 2015. Auto-completing bug reports for Android applications. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (Bergamo, Italy) (ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 673–686. <https://doi.org/10.1145/2786805.2786857>
- [46] Kevin Moran, Mario Linares-Vásquez, Carlos Bernal-Cárdenas, and Denys Poshyvanyk. 2015. Auto-completing Bug Reports for Android Applications. In *Proceedings of the Joint Meeting on Foundations of Software Engineering (FSE'15)*. 673–686.
- [47] Kevin Moran, Mario Linares-Vasquez, Carlos Bernal-Cardenas, Christopher Vendome, and Denys Poshyvanyk. 2017. CrashScope: A Practical Tool for Automated Testing of Android Applications. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering Companion (ICSE-C)*. 15–18.
- [48] Kevin Moran, Mario Linares-Vásquez, Carlos Bernal-Cárdenas, Christopher Vendome, and Denys Poshyvanyk. 2016. Automatically Discovering, Reporting and Reproducing Android Application Crashes. In *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 33–44.
- [49] Nafiseh Nikeghbal, Amir Hossein Kargaran, and Abbas Heydarnoori. 2024. Girt-model: Automated generation of issue report templates. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 407–418.
- [50] OpenAI. 2026. Introducing GPT-5.4. <https://openai.com/index/introducing-gpt-5-4/>. Accessed: 2026-03-26.
- [51] Arthur Pilone, Marco Raglianti, Michele Lanza, Fabio Kon, and Paulo Meirelles. 2025. Automatically Augmenting GitHub Issues with Informative User Reviews. In *Proceedings*. 1–12.
- [52] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *ICML'21*.
- [53] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).
- [54] Antu Saha. 2025. Studying and Automating Issue Resolution for Software Quality. *arXiv preprint arXiv:2512.10238* (2025).
- [55] Antu Saha and Oscar Chaparro. 2025. Decoding the Issue Resolution Process in Practice via Issue Report Analysis: A Case Study of Firefox. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE'25)*.
- [56] Antu Saha, Yang Song, Junayed Mahmud, Ying Zhou, Kevin Moran, and Oscar Chaparro. 2024. Toward the Automated Localization of Buggy Mobile App UIs from Bug Descriptions. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1249–1261.
- [57] Antu Saha, Mehedi Sun, and Oscar Chaparro. 2025. Automatically Identifying Solution-Related Content in Issue Report Discussions with Language Models. *arXiv preprint arXiv:2511.06501* (2025).
- [58] EG Santana Jr, Gabriel Benjamin, Melissa Araujo, Harrison Santos, David Freitas, Eduardo Almeida, Paulo Anselmo da Neto, Jiawei Li, Jina Chun, and Iftekhar Ahmed. 2025. Which Prompting Technique Should I Use? An Empirical Investigation of Prompting Techniques for Software Engineering Tasks. *arXiv preprint arXiv:2506.05614* (2025).
- [59] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, Hyojung Han, Sevien Schulhoff, et al. 2024. The prompt report: a systematic survey of prompt engineering techniques. *arXiv preprint arXiv:2406.06608* (2024).
- [60] Lin Shi, Fangwen Mu, Yumin Zhang, Ye Yang, Junjie Chen, Xiao Chen, Hanzhi Jiang, Ziyu Jiang, and Qing Wang. 2022. Buglistener: identifying and synthesizing bug reports from collaborative live chats. In *Proceedings of the 44th international conference on software engineering*. 299–311.
- [61] Kalyanasundaram Somasundaram and Gail C Murphy. 2012. Automatic categorization of bug reports using latent dirichlet allocation. In *Proceedings of the 5th India software engineering conference*. 125–130.
- [62] Yang Song and Oscar Chaparro. 2020. Bee: A tool for structuring and analyzing bug reports. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1551–1555.
- [63] Yang Song, Junayed Mahmud, Nadeeshan De Silva, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2023. BURT: A Chatbot for Interactive Bug Reporting. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE'23)*. 170–174.
- [64] Yang Song, Junayed Mahmud, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2022. Toward interactive bug reporting for (android app) end-users. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 344–356.
- [65] Xuchen Tan, Deenu Yadav, Faiz Ahmed, and Maleknaz Nayebi. 2025. ImageR: Enhancing Bug Report Clarity by Screenshots. *arXiv preprint arXiv:2505.01925* (2025).
- [66] Yuan Tian, David Lo, Xin Xia, and Chengnian Sun. 2015. Automated prediction of bug report priority using multi-factor analysis. *Empirical Software Engineering* 20 (2015), 1354–1383.
- [67] Qasim Umer, Hui Liu, and Inam Illahi. 2019. CNN-based automatic prioritization of bug reports. *IEEE Transactions on Reliability* 69, 4 (2019), 1341–1354.
- [68] Anthony J Viera, Joanne M Garrett, et al. 2005. Understanding interobserver agreement: the kappa statistic. *Fam med* 37, 5 (2005), 360–363.
- [69] Dingbang Wang, Yu Zhao, Sidong Feng, Zhaoxu Zhang, William G. J. Halfond, Chunyang Chen, Xiaoxia Sun, Jiangfan Shi, and Tingting Yu. 2024. Feedback-Driven Automated Whole Bug Report Reproduction for Android Apps. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria). 1048–1060.
- [70] Jules White, Sam Hays, Quchen Fu, Jesse Spencer-Smith, and Douglas C Schmidt. 2024. Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. In *Generative AI for Effective Software Development*. Springer, 71–108.
- [71] Yanfu Yan, Nathan Cooper, Oscar Chaparro, Kevin Moran, and Denys Poshyvanyk. 2024. Semantic GUI Scene Learning and Video Alignment for Detecting Duplicate Video-based Bug Reports. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [72] Jin Zhang, Maoqi Peng, and Yang Zhang. 2025. An Empirical Study of Transformer Models on Automatically Templating GitHub Issue Reports. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 615–626.
- [73] Yifan Zhang, Yang Yuan, and Andrew Chi-Chih Yao. 2023. Meta prompting for ai systems. *arXiv preprint arXiv:2311.11482* (2023).
- [74] Jian Zhou and Hongyu Zhang. 2012. Learning to rank duplicate bug reports. In *CIKM'12*. 852–861.
- [75] Thomas Zimmermann, Nachiappan Nagappan, Philip J. Guo, and Brendan Murphy. 2012. Characterizing and predicting which bugs get reopened. In *Proceedings of the International Conference on Software Engineering (ICSE'12)*. 1074–1083.
- [76] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schrotter, and Cathrin Weiss. 2010. What Makes a Good Bug Report? *IEEE Transactions on Software Engineering* 36, 5 (2010), 618–643. <https://doi.org/10.1109/TSE.2010.63>
- [77] Weiqin Zou, David Lo, Zhenyu Chen, Xin Xia, Yang Feng, and Baowen Xu. 2018. How practitioners perceive automated bug report management techniques. *IEEE Transactions on Software Engineering* 46, 8 (2018), 836–862.